

CS 497/597 Final Project Report

Student: Sujan Khadka

Course: CS 497/597 – Generative AI Agent Development

Date: May 2026

1. Problem & Motivation

I am an international student applying for summer internships. Like most applicants, I send out a lot of applications just to stay competitive, and I get a lot of rejections. The frustrating part is not the rejection itself. It is that every single one says the same thing: “We have decided to move forward with other candidates.” That is it. No reason, no feedback, nothing to actually work with.

At a recent career fair I asked a hiring manager about this directly. She was honest. They receive hundreds of applications per role and there is simply no time to give everyone individual feedback. I get it. But from the applicant’s side, it means I have no idea if I am being screened out because of a missing skill, weak projects, lack of experience, or something else entirely. Without knowing what the gap is, I cannot fix it.

That is what motivated this project. I wanted to build something that fixes both sides of the problem at once. On the recruiter side, it automates resume scoring and routes candidates to the right outcome. On the candidate side, when someone is rejected, they get a real explanation of why, not a form letter. And when someone is accepted, the system automatically schedules the interview and generates a tailored question set for the hiring manager based on that specific candidate’s profile. The recruiter’s only job is to make the final call.

A multi-agent setup made sense here because the work is genuinely different depending on the outcome. Scoring a resume is a different task from scheduling a calendar event, which is a different task from drafting a rejection email that cites specific skill gaps. Trying to do all of that in one prompt would either produce a giant fragile prompt or force the model to context-switch in ways that hurt quality. Breaking it into specialized agents that hand off structured data between each other keeps each step clean.

2. System Architecture

The system is built with LangGraph and uses a **two-phase, two-graph design** with a parallel fan-out on the accept path. The pipeline is split into two separate LangGraph graphs on purpose, and that split is where the human-in-the-loop actually happens.

Phase 1 — Review Graph

The recruiter opens the Streamlit app, uploads a resume PDF, selects the role, and pastes a job description. Once they click “Analyze Resume,” Phase 1 runs: a single-node graph that only contains the Review Agent. **Agent 1, the Review Agent**, retrieves the top 6 most relevant chunks from the RAG knowledge base, then calls the LLM with the resume, job description, and retrieved context all in one prompt. It returns a structured `GapAnalysis` object with seven fields: strengths, skill gaps, score (0–100), a recommendation, the deciding factor, a summary, and the candidate’s experience level. The graph ends and control returns to the Streamlit UI, which displays the gap analysis to the recruiter.

This is the human-in-the-loop point. The graph genuinely stops after Phase 1. The recruiter reads the AI’s analysis, adds optional notes, and presses either Accept or Reject. That decision is not passed back into the same graph — it is held in Streamlit session state while the recruiter reviews, then passed as input to Phase 2.

Phase 2 — Downstream Graph

Once the recruiter makes their decision, Phase 2 runs a second graph starting at the Orchestrator. **Agent 2, the Orchestrator Agent**, reads the recruiter’s decision and the gap analysis from state, logs the routing decision, and returns an empty dict — it does not call an LLM. The routing is deterministic: a conditional edge function called `route_from_orchestrator()` reads the `recruiter_decision` field and decides where to go next.

On the **accept path**, `route_from_orchestrator()` returns two `Send()` calls, which LangGraph uses to fan out to two agents simultaneously:

- **Agent 3, the Communication Agent** (ReAct pattern via `create_react_agent`), sends a personalized acceptance email to the candidate and creates a Google Calendar interview event for the next business day at 10 AM. The interview date is calculated programmatically before the agent runs so the LLM does not have to reason about dates. The agent then decides on its own when to call each tool.
- **Agent 4, the Preparation Pack Agent** (ReAct pattern), generates 12–16 tailored interview questions organized into four sections: gap-verification questions, strength-verification questions, role-critical questions, and behavioral questions. Each question includes what a strong answer looks like and what a weak one looks like. The agent then calls a file I/O tool to save the prep pack as a markdown file.

On the **reject path**, the Orchestrator routes to **Agent 5, the Rejection Agent** (ReAct pattern), which reads the gap analysis — specifically the `deciding_factor` and `skill_gaps` fields — and composes a rejection email that names the actual reason. It then calls the Gmail tool to send it.

State design: All data flows through a single `RecruitmentState` TypedDict defined in `state.py`. The `errors` field uses `Annotated[list[str], operator.add]` so that both parallel agents on the accept path can append

errors without conflicting — LangGraph merges the lists automatically. Every other field the parallel agents write to is unique to that agent, so there are no reducer conflicts.

```
START → review_agent → [recruiter reviews] → orchestrator_agent
                                     accept → communication_agent
                                     preparation_pack_agent → END
                                     reject → rejection_agent
```

The Review Agent uses a plain LangChain `llm.invoke()` call — no tool calling, just a structured JSON response it parses. The Communication Agent, Preparation Pack Agent, and Rejection Agent all use `create_react_agent` from LangGraph’s prebuilt library, which gives them a ReAct loop where they reason about when to call tools rather than being told to call them at a fixed step.

3. Tool Integration

Three tools are integrated and called by agents as needed during the pipeline.

Gmail MCP Server

This tool is used by two agents. The Communication Agent uses it to send the acceptance email after composing the message from the candidate’s profile and the job context. The Rejection Agent uses it to send the specific rejection email after reading the gap analysis output. The agent composes the full message first, then decides to invoke the tool only when the content is ready. Using a real email tool rather than just printing output means the pipeline produces an actual artifact a recruiter would use, not a simulation.

Google Calendar MCP Server

This tool is used by the Communication Agent to create the interview event. The agent determines the event title, description, and attendees from the job context before invoking the tool, and the result is a real calendar invitation sent to both the candidate and the hiring manager with a Google Meet link. This was the tool that required the most careful prompt engineering because the agent had to extract a usable meeting time from context rather than ask the recruiter for one explicitly.

4. RAG Pipeline

The RAG pipeline is integrated into Agent 1, the Review Agent, and is the primary reason the scoring is company-specific rather than generic keyword matching.

Corpus: The knowledge base contains five documents written specifically for this system. The first is a collection of anonymized profiles of five past successful hires, with the skills they had, the skills they were weak on, and the hiring manager’s notes on why they were selected anyway. The second is a set of role-specific scoring rubrics for three roles: AI/ML engineer, backend engineer, and frontend engineer, written to reflect actual team priorities rather than generic job description language. The third is a rejection case study document covering five past rejections with the specific reasons and the score threshold at which each was rejected. The fourth is full job description templates for the same three roles. The fifth is a set of 90-day onboarding outcome notes that link interview signals back to how the hire actually performed, which helps the Review Agent weight certain signals more heavily.

This corpus was chosen because the goal was to make scoring reflect real hiring standards. A model that only sees the job description will do generic keyword matching. A model that also sees that a candidate with weak Python but strong problem-solving got hired and did well will score differently, and more usefully.

Embedding and storage: Documents are chunked at the paragraph level using `RecursiveCharacterTextSplitter` with a chunk size of 500 tokens and an overlap of 50. Chunks are embedded using OpenAI `text-embedding-3-small` and stored in a local ChromaDB vector store. ChromaDB was chosen because it requires no external infrastructure and is straightforward to run locally.

Retrieval at runtime: The Review Agent embeds the incoming resume and job description together and retrieves the top five most similar chunks using cosine similarity. Those chunks are injected into the Review Agent’s prompt under a `## Hiring Context` section before it scores the resume. This means the gap analysis reflects real past decisions, not just what the model thinks a recruiter would want.

Effect on agent behavior: In practice, the retrieved context causes the Review Agent to cite specific rubric criteria when explaining the deciding factor. For example, rather than saying “the candidate lacks Python experience,” it will say “the candidate has limited Python experience and this role’s rubric weights backend scripting at the highest priority tier, which was the deciding factor.” The downstream agents, especially the Rejection Agent, benefit from this specificity because they can copy the actual gap language into the rejection email without having to re-reason about it.

5. Evaluation & Results

Example 1: Accepted Candidate

Input: Resume from a senior computer science student applying for an AI/ML engineering internship. Strong Python background, two ML projects on resume

(one with published results), familiar with PyTorch and scikit-learn. Gap: no production deployment experience, no experience with data pipelines.

Results:

- **Review Agent** scored the candidate at 82 out of 100. Strengths: strong ML fundamentals, project depth, research experience. Gaps: no deployment experience, no pipeline tooling. Deciding factor: research project quality met the rubric’s highest-weighted criterion.
- **Orchestrator** routed to the accept path and invoked the Communication Agent and Preparation Pack Agent in parallel.
- **Communication Agent** sent a personalized acceptance email referencing the candidate’s ML project by name and created a calendar event for a 45-minute technical interview with a Meet link, with invitations sent to both the candidate and the hiring manager.
- **Preparation Pack Agent** generated 14 interview questions. The technical section focused on the candidate’s ML projects and asked them to walk through architecture decisions. The gap-specific section included two questions probing deployment awareness and one on how they would approach learning a new pipeline tool, since those were the identified weaknesses.

The parallel fan-out worked well here. Both agents received the same gap analysis and produced their outputs independently without needing to wait on each other. The total latency for the accept path was lower than running them sequentially would have been.

Example 2: Rejected Candidate

Input: Resume from an applicant for a backend engineering internship. Experience listed is primarily frontend work (HTML, CSS, some React). Python listed as a skill but no backend projects shown. Applying for a role where the rubric weights server-side experience and database work at the highest tier.

Results:

- **Review Agent** scored the candidate at 31 out of 100. Strengths: frontend polish, some JavaScript depth. Gaps: no backend projects, no database experience, Python listed without supporting evidence. Deciding factor: the role rubric’s highest-weighted criteria (backend and database) were both unmet.
- **Orchestrator** routed to the reject path and invoked the Rejection Agent.
- **Rejection Agent** drafted a rejection email that cited the actual gap: the role is focused on backend and database work, the candidate’s strongest experience is in frontend, and there was no evidence of backend projects or database work in the application. The email acknowledged the candidate’s frontend skills as genuine and suggested they look for frontend or full-stack roles as a better fit. It did not say “we decided to move forward with other candidates.”

The contrast with a generic rejection is exactly what motivated this project. The candidate now knows what the gap was, which specific type of role their profile fits better, and that their skills were actually seen, not dismissed.

Where the system works well: The gap analysis object passing through state gives every downstream agent a shared, specific understanding of the candidate’s profile. The Rejection Agent does not have to re-read the resume; it reads a structured summary with a deciding factor already identified. This makes the rejection emails specific without requiring the Rejection Agent to do heavy reasoning. The RAG grounding makes the Review Agent’s output reflect real past hiring decisions rather than generic standards.

Where it still struggles: The corpus is small (five documents per category), which means retrieval for roles outside the three covered rubrics returns weak results. If a recruiter uploads a job description for a role that does not closely match any rubric, the scoring becomes less grounded. The Google Calendar tool integration also requires the agent to pick a meeting time from context, and when the recruiter does not specify a preference, the agent defaults to a time that may not be ideal. A follow-up prompt asking the recruiter for preferred times would be cleaner but would add a step to the flow. Finally, the system currently handles one candidate at a time through the Streamlit UI; there is no batch processing mode for reviewing a full applicant pool at once.

6. Reflection

The biggest design decision that paid off was keeping the gap analysis as a structured object rather than a free-text summary. Early in the project I had the Review Agent return a paragraph describing the candidate. The Rejection Agent could parse it, but the quality of the rejection emails varied a lot depending on how the paragraph was phrased. Switching to a structured object with explicit fields for strengths, gaps, score, and deciding factor made every downstream agent more reliable because they were reading a consistent format rather than interpreting prose.

The hardest part of the project was the Google Calendar integration, and it was not what I expected. I assumed the hard part would be getting the agent to construct the right event format. The actual hard part was everything before that — setting up the project on Google Cloud Platform, enabling the Calendar API, configuring OAuth credentials, downloading the credentials file, and going through the consent flow to generate a token. That process took much longer than I planned for. The Google Cloud console has a lot of steps and the documentation assumes you already know where things are. Once that was done, the actual calendar event creation worked, but I lost a significant amount of time just on the platform setup side. The Gmail integration was easier because it only needed an App Password from the Google account settings, no OAuth, no Cloud console.

Looking back, the one thing I would do differently is start with a larger corpus from the beginning. Five documents per category was enough to demonstrate the RAG pipeline works, but retrieval quality noticeably drops when the incoming job description uses different terminology than the rubric documents. Building the corpus to 20 or 30 documents covering more role variants would make the system meaningfully more robust.

If I had more time, I would add a batch processing mode that lets a recruiter upload an entire applicant pool at once and get a ranked list with gap summaries before making individual decisions. I would also add a human-in-the-loop review step where the recruiter can read the draft rejection email and approve or edit it before it sends, similar to how the sample medical project uses LangGraph's `interrupt()` function. Right now the rejection email sends automatically once the recruiter makes the reject decision, which works but gives the recruiter no chance to soften the language if they want to.

7. Tools & Acknowledgments

Development tools: Claude Code (Anthropic) was used as an AI coding assistant during development of this project. Claude Code assisted with LangGraph graph wiring, MCP tool integration, and debugging state management issues. All system design decisions, agent prompts, the RAG corpus documents, evaluation narratives, and written report prose are the author's own work.

APIs and data sources:

- Email tool: Gmail MCP server
- Calendar tool: Google Calendar MCP server
- Embeddings: OpenAI text-embedding-3-small
- Vector store: ChromaDB (local, no external infrastructure required)